5

Sistem Operasi & Cara Kerja Komputer

Membedah Peran Sistem Operasi, Penjadwalan Multitasking (Round-Robin), serta Siklus Eksekusi CPU & Mesin Konseptual

---

Modul Pembelajaran Informatika SMA Kelas X — Bab Sistem Komputer
Mengintegrasikan konsep *Berpikir Komputasional*, *Struktur Data Antrean (Queue)*, dan *Notasi Algoritma* ke dalam mekanisme kerja internal sistem komputer.



Ilustrasi kolaborasi antara Perangkat Keras (Hardware), Sistem Operasi, Aplikasi, dan Pengguna dalam Sistem Komputer.

BilikInformatika | Nur Isnanto Nugroho, S.Pd
Disusun selaras dengan Buku Siswa Informatika Kelas X (Kemendikbudristek)

Bab 5 — Sistem Operasi & Cara Kerja Komputer

Memahami bagaimana perangkat keras, perangkat lunak, dan pengguna berkolaborasi di bawah kendali Sistem Operasi

Pernah Mengalami Ini?

Mari kita mulai pembelajaran ini dari sebuah aktivitas yang hampir setiap hari kamu lakukan. Bayangkan kamu sedang duduk di depan laptop atau memegang ponsel pintar (*smartphone*). Sambil mendengarkan lagu favorit melalui aplikasi pemutar musik, kamu mengetik laporan tugas sekolah di aplikasi pengolah kata (seperti Microsoft Word), dan sesekali membalas pesan grup kelas yang masuk di layar. Saat jari-jarimu menekan tombol huruf di papan ketik (*keyboard*), huruf tersebut seketika tampil di layar monitor, alunan musik tetap terdengar jernih melalui *speaker*, dan tidak ada satu pun aplikasi yang saling berebut layar atau macet.

Pernahkah terlintas di pikiranmu, siapa "manajer" tak terlihat di balik layar yang mengatur seluruh komponen mesin dan aplikasi tersebut agar bisa bekerja serempak dengan begitu tertib? Mengapa *printer* langsung mengerti saat kamu menekan tombol *Print*, dan bagaimana satu keping prosesor (CPU) sanggup melayani puluhan program yang berjalan bersamaan?

Agar lebih mudah dipahami, bayangkan sebuah dapur restoran besar yang sedang kebanjiran pesanan. Di dapur tersebut terdapat kompor, oven, wajan, dan pisau (itulah **Perangkat Keras / *Hardware***). Ada pula buku kumpulan resep masakan standar untuk membuat berbagai hidangan lezat (itulah **Program Aplikasi / *Software***), serta pelanggan yang memesan makanan (itulah **Pengguna / *Brainware***). Apakah kompor dan buku resep bisa memasak sendiri secara otomatis? Tentu tidak! Dapur tersebut membutuhkan seorang **Kepala Koki (Sistem Operasi)** yang bertugas menyalakan kompor, membagi giliran pemakaian oven, mengambil bahan dari lemari penyimpanan, serta memastikan setiap pesanan selesai tepat waktu.

Apa Itu Sistem Komputer & Komponen Penyusunnya?

Secara mendasar, komputer adalah peralatan elektronik yang menerima masukan data (*input*) berupa teks, suara, klik, atau sentuhan, mengolah data tersebut di dalam unit pemroses pusat atau **CPU (*Central Processing Unit*)**, lalu menghasilkan keluaran (*output*) berupa informasi visual, audio, maupun sinyal kendali. Sebagaimana dijelaskan oleh Wahyono, Wisnubhadra, & Pratiwi (2021) serta Beekman (2012), sebuah **Sistem Komputer** wajib disusun oleh tiga komponen utama yang saling mendukung:

- **Perangkat Keras (*Hardware*) — "Tubuh & Organ Fisik"**: Komponen fisik pada komputer yang memiliki wujud nyata sehingga dapat dilihat, disentuh, atau dipindahkan. Contohnya meliputi piranti masukan (*mouse, keyboard, scanner, webcam*), unit pemroses dan memori (*processor/CPU, RAM, ROM, harddisk*), serta piranti keluaran (*monitor, printer, speaker*). Tanpa perangkat lunak, perangkat keras hanyalah benda mati layaknya tubuh manusia tanpa jiwa.

- **Perangkat Lunak (*Software*) — "Jiwa & Pikiran"**: Kumpulan kode program atau instruksi sistematis yang dibuat menggunakan bahasa pemrograman. *Software* tidak dapat disentuh secara fisik, tetapi dioperasikan melalui antarmuka untuk menjembatani keinginan pengguna dengan perangkat keras. Contohnya adalah **Sistem Operasi** (MS Windows, Linux, macOS, Android, iOS) dan **Program Aplikasi** (MS Office, peramban web, aplikasi permainan).
- **Pengguna (*Brainware / User*) — "Sang Pengendali"**: Manusia yang mengoperasikan, memberikan instruksi, serta memanfaatkan hasil komputasi dari sistem komputer dalam kehidupan sehari-hari.

Ragam Ukuran Komputer & Cara Manusia Berinteraksi

Berdasarkan ukuran fisik dan kapasitas pengolahannya (Wahyono dkk., 2021; Beekman, 2012), komputer dikelompokkan mulai dari yang terkecil hingga yang paling raksasa: (1) **Microcomputer (Komputer Mikro)** seperti *smartphone*, tablet, konsol gim, laptop/*ultrabook*, hingga komputer papan tunggal seukuran kartu kredit seperti *Raspberry Pi* dan *Arduino* (untuk proyek IoT dan robotika); (2) **Personal Computer (PC) & Mini PC** untuk kebutuhan personal maupun kantor kecil; (3) **Minicomputer & Komputer Mainframe** yang bertindak sebagai peladen (*server*) perusahaan berskala menengah hingga besar (seperti IBM z Systems); serta (4) **Supercomputer** (seperti Sierra/ATS-2 milik NASA) yang mampu mengeksekusi triliunan instruksi per detik dalam satuan **FLOPS (*Floating Point Operation per Second*)**.

Untuk berkomunikasi dengan berbagai jenis komputer tersebut, pengguna menggunakan beberapa mekanisme **Antarmuka (*Interface*)**. Dua jenis antarmuka perangkat lunak yang paling utama adalah **GUI (*Graphical User Interface*)** dan **CLI (*Command Line Interface*)**, ditambah interaksi melalui **Suara (*Audio*)**, **Gambar (*Video*)**, serta piranti masukan langsung.

Jenis Antarmuka	Kata Kunci & Karakteristik	Mekanisme Interaksi Pengguna	Contoh Penerapan Nyata
GUI (*Graphical User Interface*)	Visual, Ikon, Menu Grafis, Mudah bagi Pemula	Pengguna mengklik ikon, memilih menu tarik-turun, menekan tombol (*button*), mengisi *textbox*, memilih *radio button* (satu pilihan) atau *checkbox* (banyak pilihan).	Layar utama MS Windows, macOS, ikon aplikasi di Android/iOS, serta kotak dialog penyimpanan berkas (*Save As*).
CLI (*Command Line Interface*)	Berbasis Teks, *Text-Terminal*, Baris Perintah	Pengguna mengetikkan instruksi teks secara spesifik baris demi baris menggunakan papan ketik tanpa bantuan ikon grafis.	*Command Prompt* (CMD) atau PowerShell pada Windows, serta *Terminal* pada sistem operasi Linux (`uname -a`).
Audio, Video & Piranti Fisik	Suara, Kamera, Sentuhan & Sensor	Perekaman gelombang suara melalui mikrofon, tangkapan visual melalui *webcam*, serta gerakan fisik via *mouse, touchpad, joystick*, atau layar sentuh.	Perintah suara asisten virtual, panggilan video daring, papan sirkuit kreatif *Makey Makey*, dan sensor sistem otonom / IoT.

Kolaborasi Sistem & Peran Vital Driver Perangkat (Device Driver)

Di dalam sistem komputer, perangkat keras tidak pernah bekerja sendirian. Ketika kamu mengambil foto menggunakan *webcam* lalu mencetaknya ke mesin *printer*, apakah *webcam* mengobrol langsung dengan

printer? Tidak! *Webcam* mengirimkan data gambar ke memori komputer melalui Sistem Operasi, aplikasi pengolah gambar membukanya, lalu Sistem Operasi meneruskan perintah cetak ke *printer*.

Agar Sistem Operasi dapat mengenali perangkat keras baru yang dicolokkan (seperti *printer*, *scanner*, atau *flashdisk*), dibutuhkan komponen perangkat lunak sistem khusus yang disebut **Driver Perangkat (*Device Driver*)** (Wahyono dkk., 2021). *Driver* bertugas sebagai "penerjemah bahasa" yang menggunakan kode biner untuk menyampaikan instruksi dari Sistem Operasi kepada perangkat keras dan mengambil kembali data dari perangkat keras tersebut.

Mengenal Sistem Operasi, "Lapisan Bawang", & Proses Booting

Sistem Operasi (*Operating System* / OS) adalah perangkat lunak sistem paling fundamental yang bertugas mengelola seluruh perangkat keras komputer, mengatur sumber daya perangkat lunak, serta menyediakan layanan umum bagi program aplikasi. Tanpa Sistem Operasi, pengguna harus menulis kode biner yang sangat rumit hanya untuk menampilkan satu huruf di layar.

Dalam ilmu Informatika, posisi Sistem Operasi di dalam komputer sering digambarkan melalui model **"Lapisan Bawang" (*Onion Layer*)** (Wahyono dkk., 2021; Operating System, 2021). Perangkat keras (*Hardware*) berada di inti terdalam, dibungkus oleh **Kernel OS** (inti Sistem Operasi) beserta *Driver Perangkat*, lalu dilapisi oleh manajer memori, sistem berkas, serta antarmuka perintah (*Shell* dan *Daemon*), kemudian dilapisi oleh **Program Aplikasi**, hingga akhirnya berhadapan langsung dengan **Pengguna** di lapisan paling luar.



Gambar 5.1: (Kiri) Struktur "Lapisan Bawang" Sistem Komputer; (Kanan) Lima tahap berurutan proses Booting saat komputer dinyalakan.

Tahukah Kamu? Teka-Teki "Ayam dan Telur" saat Menyalakan Komputer (Booting)

Semua program komputer—termasuk Sistem Operasi—wajib dimuat ke dalam memori utama (RAM) dan dieksekusi oleh CPU agar bisa hidup. Namun, saat komputer baru saja dinyalakan dari keadaan mati total, RAM dan CPU masih kosong melompong! Lalu, siapa yang pertama kali membangunkan dan memuat Sistem Operasi ke dalam CPU? Teka-teki klasik yang disebut *"chicken and egg problem"* ini diselesaikan melalui algoritma penyalaan awal yang baku bernama **Booting** (Wahyono dkk., 2021; Beekman, 2012):

1. **Daya (*Power*) Dihidupkan**: Pengguna menekan tombol *Power*, sehingga arus listrik mengalir dan membangunkan seluruh komponen perangkat keras di dalam komputer.
2. **POST (*Power On Self Test*)**: Perangkat keras melakukan uji diagnostik mandiri secara cepat untuk memeriksa apakah komponen vital seperti RAM, *keyboard*, dan media penyimpanan terpasang dan berfungsi tanpa cacat.
3. **Memuat BIOS (*Basic Input Output System*)**: CPU membaca program kecil bernama BIOS yang telah ditanam secara permanen oleh pabrik di dalam chip memori **ROM (*Read-Only Memory*)** yang isinya tidak hilang meski listrik mati.
4. **Memuat Sistem Operasi**: BIOS mencari keberadaan Sistem Operasi (seperti Windows atau Linux) di penyimpanan sekunder (*harddisk* atau SSD), lalu menyalin inti Sistem Operasi tersebut ke dalam memori utama (RAM).
5. **Pengalihan Kontrol ke Sistem Operasi**: Setelah Sistem Operasi aktif di RAM, BIOS menyerahkan kendali penuh komputer kepada Sistem Operasi hingga layar *Login* atau *Desktop* siap digunakan oleh pengguna.

10 Fungsi Utama Sistem Operasi yang Wajib Kamu Kuasai

Layaknya seorang manajer gedung yang serbabisa, Sistem Operasi dibagi menjadi komponen-komponen spesifik (*dekomposisi* — Mushthofa dkk., 2021) untuk menjalankan **10 fungsi utama** berikut (Wahyono dkk., 2021; Beekman, 2012):

No	Fungsi Sistem Operasi	Kata Kunci	Penjelasan & Contoh Kerja Nyata di Komputer
1	**Menyediakan Antarmuka**	GUI, CLI, Interaksi	Menyediakan tampilan visual (ikon, jendela, menu) atau terminal teks agar manusia mudah mengoperasikan komputer.
2	**Mengendalikan Input & Output**	Kontrol I/O, Sinyal Data	Mengatur lalu lintas data dari alat masukan (ketikan *keyboard*, klik *mouse*) menuju layar monitor atau *printer*.
3	**Mengelola Perangkat Keras & Periferal**	Manajemen *Hardware*, *Driver*	Menjadi perantara antara aplikasi dan *hardware*. Saat program meminta input variabel (`scanf`/`input`), OS menunggu ketikan *keyboard* pengguna.
4	**Mengelola Pemuatan Perangkat Lunak**	Eksekusi Aplikasi, Kompatibilitas	Memuat dan menjalankan aplikasi di atas OS sehingga satu aplikasi yang sama bisa berjalan di berbagai merek perangkat keras yang berbeda.
5	**Mengelola Berkas (*File*) & Direktori**	*Copy, Save, Sort, Delete*, Tabel Pencarian	Mengatur pengarsipan *file* dan *folder* serta memelihara **Tabel Pencarian** yang menghubungkan nama berkas dengan alamat lokasi penyimpanannya.

No	Fungsi Sistem Operasi	Kata Kunci	Penjelasan & Contoh Kerja Nyata di Komputer
6	**Menangani Interupsi & Kesalahan (*Error*)**	Sinyal *Interrupt*, Prioritas, Deteksi *Bug*	Mengatur sinyal selaan (*interupsi*) reguler agar CPU dapat bergantian melayani banyak program dan menangani *error* tanpa membuat sistem macet.
7	**Mengelola Prosesor (CPU)**	*Scheduler*, Penggiliran Waktu CPU	Memuat program aktif ke RAM dan menugaskan bagian bernama **Scheduler (Penjadwal)** untuk membagi giliran waktu eksekusi CPU.
8	**Mengelola Memori Utama (RAM)**	Alokasi Memori, Pelacakan Ruang	Melacak pemakaian RAM, memutuskan jatah memori tiap aplikasi, dan mengatur ruang cadangan jika kapasitas RAM mulai penuh.
9	**Mengelola Keamanan Sistem**	Akun, Kata Sandi, *Firewall*	Mengatur otentikasi *login* pengguna (*username & password*) serta mengaktifkan program utilitas keamanan seperti *firewall*.
10	**Menangani Komunikasi Jaringan**	Protokol TCP/IP, Pertukaran Data	Mengelola koneksi perangkat dengan jaringan lokal (LAN) maupun internet agar komputer dapat mengirim dan menerima paket data.

Multitasking & Algoritma Penjadwalan Proses (Round-Robin)

Dalam kehidupan sehari-hari, kamu pasti pernah melakukan **Multitasking**—misalnya mengerjakan PR sambil mendengarkan lagu, lalu tiba-tiba berhenti sejenak karena dipanggil ibu untuk makan malam. Panggilan ibu yang membuatmu menghentikan sementara pekerjaan yang sedang berjalan itulah yang dalam ilmu komputer disebut sebagai **Interupsi (*Interrupt*)**.

Komputer juga terlihat sangat pandai melakukan *multitasking*: kamu bisa membuka MS Word, MS Excel, MS PowerPoint, dan aplikasi Paint secara bersamaan sambil menyalin teks melalui *clipboard*, sementara jam di pojok layar terus berdetak dan notifikasi *email* tetap masuk. Namun, tahukah kamu rahasia di baliknya? Sebuah prosesor (CPU) tunggal sebenarnya **hanya mampu mengeksekusi satu instruksi pada satu waktu**! Komputer tampak mengerjakan semuanya sekaligus karena komponen **Scheduler (Penjadwal Proses)** di dalam Sistem Operasi membagi waktu CPU secara bergiliran dalam hitungan milidetik menggunakan struktur data **Antrean (*Queue* - FIFO)** (Mushthofa dkk., 2021; Queue, 2021) dan algoritma bernama **Round-Robin (RR)** (Wahyono dkk., 2021).

Cara Kerja Algoritma Penjadwalan Round-Robin (RR)

Algoritma **Round-Robin (RR)** bekerja dengan prinsip keadilan melingkar tanpa memandang prioritas. Penjadwal menetapkan satu jatah potongan waktu tetap yang disebut **Kuantum (*Time Slice*)**, misalnya **100 milidetik (ms)**. Setiap pekerjaan (*job* atau proses) yang datang akan dimasukkan ke baris belakang **Antrean Siap (*Ready Queue* — konsep *Enqueue*)**, dan proses di posisi paling depan akan diambil untuk dieksekusi oleh CPU (konsep *Dequeue*). Terdapat dua pendekatan algoritma saat menjalankan *Round-Robin*:

- **Algoritma Pertama (CPU Tidak Pernah Berhenti — Paling Efisien)**:
  1. Selama periode satu slot waktu (1 Kuantum): jika suatu proses selesai sebelum atau tepat saat kuantum habis, langsung hapus proses tersebut dari antrean dan **seketika ambil proses berikutnya** tanpa menunggu sisa kuantum habis.
  2. Di akhir satu kuantum: jika proses yang sedang dikerjakan masih belum selesai (sisa waktu eksekusi > 0), sistem melakukan **interupsi**, memasukkan kembali proses tersebut ke urutan paling belakang antrean (*re-queue*), lalu memberikan giliran CPU kepada proses terdepan berikutnya.
- **Algoritma Kedua (Tunggu Akhir Kuantum)**: Jika ada proses yang selesai lebih cepat di tengah kuantum, hapus dari antrean tetapi CPU menunggu (menganggur) sampai batas kuantum berakhir, baru mengambil giliran berikutnya.

Kata Kunci Penjadwalan Round-Robin:
Slot Waktu Sama Rata (Kuantum / Time Slice), Antrean Melingkar (Queue FIFO), Interupsi jika belum selesai, Masukkan kembali ke belakang antrean.

Contoh Soal Simulasi 1 — Penjadwalan Proses Round-Robin (Kuantum = 100 ms)

Kasus Simulasi (Mengacu Tabel 4.1 Buku Informatika Kelas X):
Sebuah Sistem Operasi menggunakan algoritma penjadwalan **Round-Robin Pertama (CPU tidak pernah berhenti)** dengan jatah **Kuantum = 100 ms**. Terdapat 6 buah proses (P0 sampai P5) yang datang ke dalam sistem dengan rincian waktu kedatangan dan kebutuhan waktu eksekusi sebagai berikut:

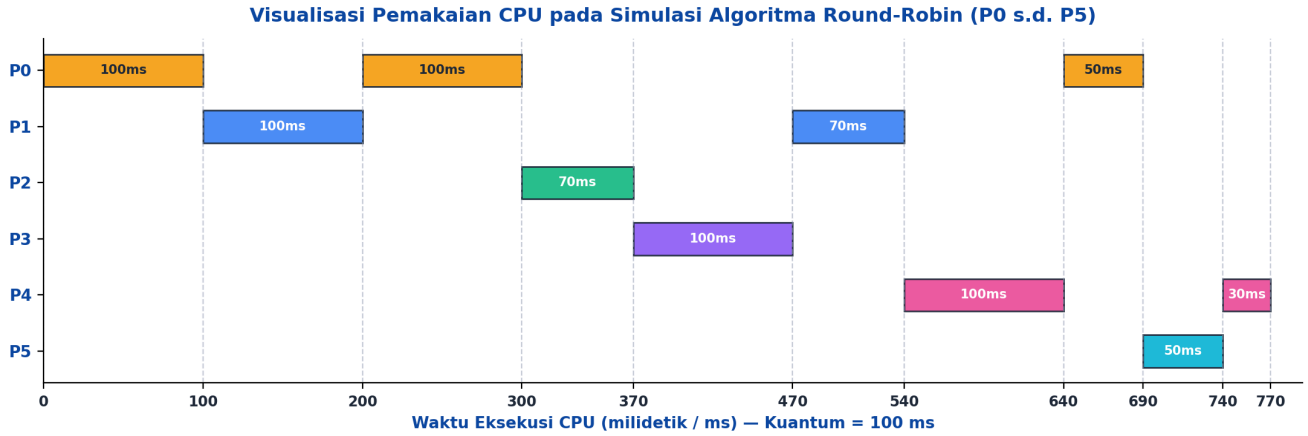
Nama Proses	Waktu Kedatangan (ms)	Waktu Eksekusi yang Dibutuhkan (ms)	Keterangan Jatah Kuantum (100 ms)
P0	0 ms	250 ms	Butuh 3 kali giliran di CPU (100 ms + 100 ms + 50 ms)
P1	50 ms	170 ms	Butuh 2 kali giliran di CPU (100 ms + 70 ms)
P2	120 ms	70 ms	Selesai hanya dalam 1 kali giliran (70 ms < 100 ms)
P3	170 ms	100 ms	Selesai tepat dalam 1 kali giliran (100 ms)
P4	200 ms	130 ms	Butuh 2 kali giliran di CPU (100 ms + 30 ms)
P5	350 ms	50 ms	Selesai hanya dalam 1 kali giliran (50 ms < 100 ms)
Total	—	**770 ms**	Total waktu kerja CPU tanpa henti dari 0 ms s.d. 770 ms

Pertanyaan Analisis:
(A) Bagaimana jejak antrean dan eksekusi CPU dari waktu `0 ms` hingga `770 ms`?
(B) Proses manakah yang selesai paling pertama, dan proses manakah yang selesai paling terakhir?

Pembahasan Langkah demi Langkah (Penelusuran Tabel Antrean & CPU):

Waktu (ms)	Sedang Diproses CPU	Isi Antrean Tunggu (Depan → Belakang)	Proses Selesai	Penjelasan Logika Kejadian pada Sistem Operasi
0	**P0** (sisa 250)	*(Kosong)*	—	P0 datang pada 0 ms dan langsung dieksekusi oleh CPU.
50	**P0** (sisa 200)	[P1(170)]	—	P1 datang pada 50 ms dan masuk ke antrean tunggu.
100	**P1** (sisa 170)	[P0(150)]	—	Kuantum 100 ms P0 habis (sisa 150 ms). P0 diinterupsi & masuk antrean belakang; P1 maju ke CPU.
120 & 170	**P1** (sedang jalan)	[P0(150), P2(70), P3(100)]	—	Pada 120 ms P2 datang; pada 170 ms P3 datang. Keduanya berbaris di belakang P0.
200	**P0** (sisa 150)	[P2(70), P3(100), P1(70), P4(130)]	—	Kuantum P1 habis (sisa 70 ms) dan diantrekan kembali bersama P4 yang baru datang. P0 maju ke CPU.
300	**P2** (sisa 70)	[P3(100), P1(70), P4(130), P0(50)]	—	Kuantum kedua P0 habis (sisa tinggal 50 ms) dan kembali ke belakang antrean. Giliran P2 masuk CPU.
350	**P2** (sisa 20)	[P3(100), P1(70), P4(130), P0(50), P5(50)]	—	Pada 350 ms P5 datang dan menempati barisan paling akhir antrean.
370	**P3** (sisa 100)	[P1(70), P4(130), P0(50), P5(50)]	**P2**	P2 hanya butuh 70 ms sehingga **selesai tepat pada 370 ms**! CPU langsung menarik P3 tanpa menunggu 400 ms.
470	**P1** (sisa 70)	[P4(130), P0(50), P5(50)]	**P3**	Setelah berjalan 100 ms (370 → 470 ms), **P3 selesai**! Giliran berikutnya adalah P1.
540	**P4** (sisa 130)	[P0(50), P5(50)]	**P1**	Sisa 70 ms milik P1 tuntas pada 540 ms (**P1 selesai**). CPU langsung mengeksekusi P4 selama 100 ms.
640	**P0** (sisa 50)	[P5(50), P4(30)]	—	Kuantum pertama P4 habis pada 640 ms (sisa 30 ms) dan diantrekan di belakang P5. P0 maju ke CPU.

Waktu (ms)	Sedang Diproses CPU	Isi Antrean Tunggu (Depan → Belakang)	Proses Selesai	Penjelasan Logika Kejadian pada Sistem Operasi
690	**P5** (sisa 50)	[P4(30)]	**P0**	Sisa 50 ms milik P0 tuntas pada 690 ms (**P0 selesai**). CPU langsung mengerjakan P5.
740	**P4** (sisa 30)	*(Kosong)*	**P5**	P5 tuntas dalam 50 ms pada 740 ms (**P5 selesai**). Tinggal P4 yang masuk ke CPU untuk menghabiskan sisa 30 ms.
770	*(Selesai)*	*(Kosong)*	**P4**	Seluruh eksekusi selesai tepat pada **770 ms**!



Gambar 5.2: Diagram garis waktu (Gantt Chart) pemakaian CPU pada simulasi Round-Robin dengan Kuantum 100 ms.

Kesimpulan Jawaban Simulasi 1:
Urutan proses yang selesai dari paling awal hingga paling akhir adalah: **P2 (370 ms) → P3 (470 ms) → P1 (540 ms) → P0 (690 ms) → P5 (740 ms) → P4 (770 ms)**. Perhatikan bahwa meskipun P0 datang paling pertama pada detik ke-0, karena ukuran pekerjaannya besar (250 ms), ia baru selesai pada 690 ms, sedangkan P2 yang ukurannya kecil (70 ms) selesai paling pertama!

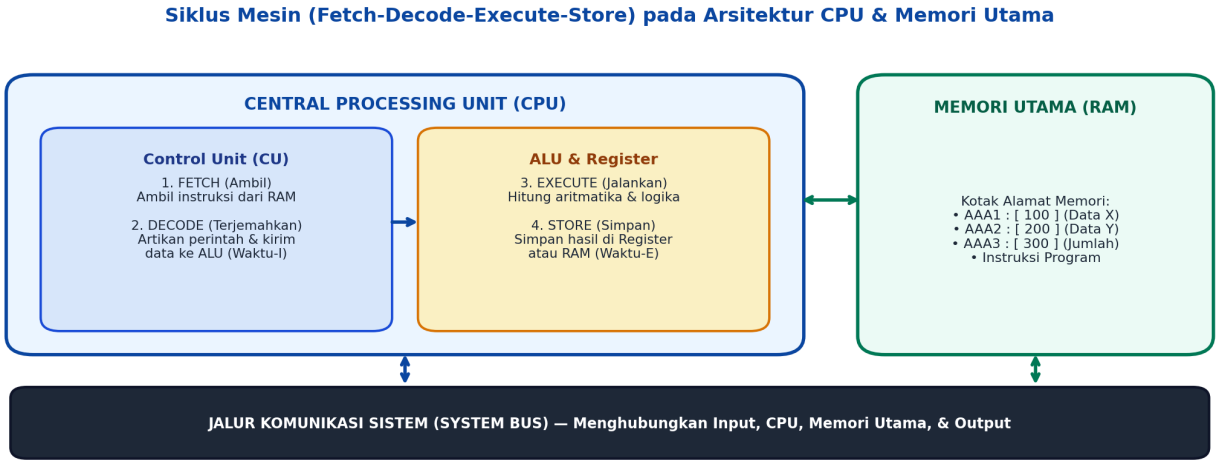
Cara Kerja Internal CPU, Memori, & Mesin Konseptual Mr. ALGO

Setelah memahami bagaimana Sistem Operasi menjadwalkan program, sekarang mari kita membedah bagaimana keping prosesor atau **CPU (*Central Processing Unit*)** mengeksekusi setiap baris instruksi program bersama **Memori Utama (RAM)** berdasarkan arsitektur mesin komputer (Vic, 2005; Wahyono dkk., 2021).

- **Dua Bagian Utama di Dalam CPU**:
  1. **CU (*Control Unit* / Unit Kontrol)**: Bertugas mengatur seluruh sinyal kontrol prosesor, mengambil instruksi dari memori utama, menerjemahkan arti instruksi tersebut, dan memerintahkan perpindahan data.
  2. **ALU (*Arithmetic/Logic Unit* / Unit Aritmatika dan Logika)**: Bertugas sebagai "kalkulator super cepat" yang menjalankan operasi perhitungan matematika (tambah, kurang, kali, bagi) dan

perbandingan logika, dibantu oleh ruang simpan internal CPU berkecepatan tinggi bernama **Register**.

- **Memori Utama (Primer) vs Memori Sekunder & Analogi Kotak Surat**: Memori utama (RAM) menyimpan data dan instruksi secara *sementara* hanya saat program sedang dieksekusi oleh CPU, sedangkan memori sekunder (*harddisk*, SSD, CD-ROM, *flashdisk*) menyimpan data secara *permanen*. Lokasi penyimpanan di dalam RAM diibaratkan seperti deretan **loker atau kotak surat bernomor tetap** yang disebut **Alamat Memori (*Memory Address*)**. Isi di dalam kotak bisa berganti-ganti (diukur dalam satuan *byte*), namun nomor alamat kotaknya selalu tetap. Agar programmer tidak pusing menghafal kode alamat mesin (seperti `AAA1` atau `Kotak 3`), bahasa pemrograman menggunakan **Alamat Simbolis (Variabel)** seperti `Tarif`, `Jam`, dan `Gaji`.
- **Clock Internal & Instruction Set**: CPU memiliki detak jantung elektronik bernama **Clock Internal** yang menghasilkan pulsa (*pulse*) berkecepatan tetap (jutaan hingga miliaran detak per detik) untuk menyinkronkan setiap langkah kerja. Setiap jenis CPU dirancang memahami kosakata perintah mesin tertentu yang disebut **Instruction Set**.



Gambar 5.3: Empat tahapan Siklus Mesin (Fetch-Decode-Execute-Store) yang berulang terus-menerus selama komputer hidup.

Siklus Ambil dan Jalankan (Fetch-Decode-Execute Cycle / Siklus Mesin)

Menurut Vic (2005) dan Wahyono dkk. (2021), setiap kali menjalankan satu instruksi program, CPU dan Memori Utama berkolaborasi melalui **4 langkah Siklus Mesin** yang terbagi atas **Waktu Instruksi (Waktu-I)** dan **Waktu Eksekusi (Waktu-E)**:

1. **Langkah 1 — FETCH (Ambil Instruksi) [Waktu-I]**: Unit Kontrol (CU) mengambil satu baris instruksi program yang tersimpan di dalam alamat Memori Utama (RAM).
2. **Langkah 2 — DECODE (Terjemahkan Instruksi) [Waktu-I]**: Unit Kontrol (CU) menerjemahkan arti perintah tersebut dan memerintahkan agar data yang dibutuhkan disalin dari Memori Utama menuju ke Register/ALU.
3. **Langkah 3 — EXECUTE (Jalankan Operasi) [Waktu-E]**: Unit Aritmatika/Logika (ALU) mengambil alih kendali dan mengeksekusi perhitungan matematika atau evaluasi logika pada data di dalam Register.
4. **Langkah 4 — STORE (Simpan Hasil) [Waktu-E]**: Hasil perhitungan dari ALU disimpan kembali ke dalam Register atau alamat Memori Utama, untuk selanjutnya dikirimkan ke perangkat keluaran

(seperti layar monitor).

Simulasi Mesin Konseptual Sederhana Ciptaan Mr. ALGO

Bagaimana kode program tingkat tinggi seperti `X = 100; Y = 200; Jumlah = X + Y; PRINT Jumlah` dijalankan di tingkat mesin? Karena ALU tidak bisa menjumlahkan data yang masih berada di dalam keping RAM, data tersebut wajib disalin terlebih dahulu dari alamat RAM ke **Register CPU**! Dalam buku *Informatika Kelas X* (Wahyono dkk., 2021; Ramadhan dkk., 2021), mekanisme translasi instruksi ini dipelajari melalui **Mesin Konseptual Sederhana ciptaan Mr. ALGO** yang memiliki variasi arsitektur register:

Model Mesin Mr. ALGO	Spesifikasi Register & Memori	Aturan Penyimpanan Hasil Operasi Aritmatika	Daftar Perintah Bahasa Mesin
Mesin 3-Register (Aktivitas SK-K10-02-U)	5 Alamat Memori (AEB1 s.d. AEB5) & 3 Register (REG1, REG2, REG3).	Operasi pada REG1 dan REG2 selalu menyimpan hasilnya di **REG3**.	`SIMPAN <data> <alamat>`, `SALIN <asal> <tujuan>`, `TAMBAH/KURANG/KALI/BAGI REG1 REG2`, `PRINT <alamat>`.
Mesin 2-Register (Mesin-1 vs Mesin-2)	Alamat Memori (AAA1 s.d. AAA4) & 2–3 Register (REG1, REG2, REG3).	**Mesin-1**: Hasil disimpan di register operan pertama. **Mesin-2**: Hasil disimpan di register operan kedua.	Contoh `TAMBAH REG1 REG2`: • Pada Mesin-1 masuk ke REG1. • Pada Mesin-2 masuk ke REG2.
Mesin 1-Register (Mesin Akumulator)	4 Alamat Memori (AAA1 s.d. AAA4) & 1 Register tunggal bernama **AKUMULATOR**.	Semua operan dan hasil perhitungan langsung ditampung di dalam **AKUMULATOR**.	`INPUT <data> <alamat>`, `LOAD <alamat>` (salin ke Akumulator), `TAMBAH/KURANG/KALI/BAGI <alamat>`, `OUTPUT`.

Contoh Soal Simulasi 2 — Menerjemahkan Operasi Matematika ke Mesin Mr. ALGO

Soal A (Mesin 3-Register Mr. ALGO — Hasil di REG3):
Terjemahkan operasi matematika **3 + 8 × 9** ke dalam urutan instruksi bahasa mesin Mr. ALGO (memori AEB1–AEB5, register REG1–REG3) dan lacak isi memori serta registernya!

Pembahasan Soal A: Ingat aturan prioritas operasi matematika (*operator precedence*): perkalian 8 × 9 wajib dihitung lebih dulu, baru hasilnya dijumlahkan dengan 3.

Langkah & Instruksi Mesin	Penjelasan Aksi CPU & Memori	Isi Memori (AEB1, AEB2, AEB3)	Isi Register (REG1, REG2, REG3)
`1. SIMPAN 3 AEB1`	Simpan angka 3 ke kotak memori AEB1.	AEB1=3, AEB2=-, AEB3=-	REG1=-, REG2=-, REG3=-

Langkah & Instruksi Mesin	Penjelasan Aksi CPU & Memori	Isi Memori (AEB1, AEB2, AEB3)	Isi Register (REG1, REG2, REG3)
`2. SIMPAN 8 AEB2`	Simpan angka 8 ke kotak memori AEB2.	`AEB1=3, AEB2=8, AEB3=-`	`REG1=-, REG2=-, REG3=-`
`3. SIMPAN 9 AEB3`	Simpan angka 9 ke kotak memori AEB3.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=-, REG2=-, REG3=-`
`4. SALIN AEB2 REG1`	Salin angka 8 dari AEB2 ke REG1.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=8, REG2=-, REG3=-`
`5. SALIN AEB3 REG2`	Salin angka 9 dari AEB3 ke REG2.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=8, REG2=9, REG3=-`
`6. KALI REG1 REG2`	ALU mengalikan `8 × 9 = 72`, hasil disimpan di REG3.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=8, REG2=9, REG3=72`
`7. SALIN AEB1 REG1`	Salin angka 3 dari AEB1 ke REG1.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=3, REG2=9, REG3=72`
`8. SALIN REG3 REG2`	Pindahkan hasil kali (72) dari REG3 ke REG2.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=3, REG2=72, REG3=72`
`9. TAMBAH REG1 REG2`	ALU menjumlahkan `3 + 72 = 75`, hasil masuk ke REG3.	`AEB1=3, AEB2=8, AEB3=9`	`REG1=3, REG2=72, REG3=75`
`10. SALIN REG3 AEB4`	Simpan hasil akhir (75) dari REG3 ke memori AEB4.	`AEB1=3..AEB3=9,` **`AEB4=75`**	`REG1=3, REG2=72, REG3=75`
`11. PRINT AEB4`	Cetak isi memori AEB4 ke layar monitor (**Output: 75**).	**Layar Monitor: 75**	Selesai dieksekusi

Soal B (Uji Kompetensi Buku Kelas X Hal. 89 — Mesin-1 vs Mesin-2 & Mesin Akumulator):
1. Bandingkan hasil akhir eksekusi deretan instruksi pada Tabel Uji Kompetensi halaman 89 antara **Mesin-1** (hasil disimpan di operan pertama) dan **Mesin-2** (hasil disimpan di operan kedua)!
2. Tuliskan urutan instruksi paling hemat memori pada **Mesin 1-Akumulator** untuk menghitung **`(1 + 4) × (8 - 5) / 5`**!

Pembahasan Soal B.1 (Perbandingan Jejak Mesin-1 vs Mesin-2):
Setelah 6 instruksi awal (`SIMPAN 8 AAA1`, `SIMPAN 9 AAA2`, `SIMPAN 10 AAA3`, lalu `SALIN` ketiganya ke `REG1`, `REG2`, `REG3`), kondisi awal kedua mesin sama persis: `AAA1=8, AAA2=9, AAA3=10` dan `REG1=8, REG2=9, REG3=10`. Perbedaan mencolok muncul saat instruksi aritmatika dijalankan:

Urutan Instruksi Lanjutan	Jejak Eksekusi pada MESIN 1 (Hasil di Operan ke-1)	Jejak Eksekusi pada MESIN 2 (Hasil di Operan ke-2)
Kondisi setelah 6 instruksi awal	`AAA=[8, 9, 10]` | `REG=[8, 9, 10]`	`AAA=[8, 9, 10]` | `REG=[8, 9, 10]`
`7. TAMBAH REG1 REG2`	`8 + 9 = 17` → masuk REG1. `REG = [17, 9, 10]`	`8 + 9 = 17` → masuk REG2. `REG = [8, 17, 10]`
`8. TAMBAH REG2 REG3`	`9 + 10 = 19` → masuk REG2. `REG = [17, 19, 10]`	`17 + 10 = 27` → masuk REG3. `REG = [8, 17, 27]`
`9. KALI REG2 REG3`	`19 × 10 = 190` → masuk REG2. `REG = [17, 190, 10]`	`17 × 27 = 459` → masuk REG3. `REG = [8, 17, 459]`
`10. KALI REG1 REG3`	`17 × 10 = 170` → masuk REG1. `REG = [170, 190, 10]`	`8 × 459 = 3672` → masuk REG3. `REG = [8, 17, 3672]`

`11. SALIN REG1 AAA3`
`12. SALIN REG2 AAA2`
`13. SALIN REG3 AAA1`
	Hasil Akhir Memori Mesin 1:
	`AAA1 = 10, AAA2 = 190, AAA3 = 170`
		Hasil Akhir Memori Mesin 2:
		`AAA1 = 3672, AAA2 = 17, AAA3 = 8`

Pembahasan Soal B.2 (Efisiensi Memori pada Mesin 1-Akumulator untuk `(1 + 4) × (8 - 5) / 5`):
Agar penggunaan memori **se-efisien mungkin**, kita tidak perlu memakai keempat kotak memori (AAA1–AAA4). Kita cukup menggunakan **2 kotak memori saja (AAA1 dan AAA2)** dengan cara menimpa kembali (*overwrite*) kotak memori yang sudah tidak terpakai:

1. `INPUT 8 AAA1` dan `INPUT 5 AAA2` → Memori: `AAA1=8, AAA2=5`.
2. `LOAD AAA1` lalu `KURANG AAA2` → Akumulator menghitung `8 - 5 = 3`.
3. `INPUT 1 AAA1` dan `INPUT 4 AAA2` → Sementara kita butuh menyimpan angka 3, atau lebih cerdas lagi: hitung `8 - 5 = 3` di awal jika ada perintah simpan ke memori, atau karena mesin Akumulator pada halaman 86 memiliki instruksi `INPUT`, `LOAD`, `TAMBAH`, `KURANG`, `KALI`, `BAGI`, dan `OUTPUT`, kita simpan konstanta 4, 5, dan hasil antara secara berurutan sehingga `(1 + 4) = 5`, dikali `(8 - 5) = 3` menghasilkan 15, lalu dibagi 5 menghasilkan **Output Akhir = 3**!

Glosarium Sistem Operasi & Sistem Komputer

Istilah Penting	Definisi Ringkas & Konsep Kunci
ALU (*Arithmetic/Logic Unit*)	Komponen utama di dalam CPU yang bertugas melakukan seluruh perhitungan aritmatika (tambah, kurang, kali, bagi) dan perbandingan logika.
BIOS (*Basic Input Output System*)	Program dasar yang tertanam di memori ROM untuk menginisialisasi perangkat keras dan memuat Sistem Operasi saat proses *booting*.

Istilah Penting	Definisi Ringkas & Konsep Kunci
Booting	Urutan proses penyalaan awal komputer (*startup*) untuk membangunkan perangkat keras dan mengaktifkan Sistem Operasi (Power → POST → BIOS → Muat OS → Alihkan Kontrol).
CLI (*Command Line Interface*)	Antarmuka pengguna berbasis teks (*text-terminal*) di mana instruksi dijalankan dengan mengetikkan baris perintah.
CU (*Control Unit*)	Unit kendali di dalam CPU yang bertugas mengambil instruksi dari memori (*Fetch*), menerjemahkannya (*Decode*), dan mengatur sinyal kontrol antarperangkat.
Device Driver (Pemacu Peranti)	Perangkat lunak sistem yang bertindak sebagai penerjemah komunikasi (dalam kode biner) antara Sistem Operasi dan perangkat keras tertentu (misalnya *driver printer*).
Fetch-Execute Cycle (Siklus Mesin)	Siklus operasi paling mendasar pada CPU dalam menjalankan program yang terdiri atas tahap *Fetch* (ambil), *Decode* (terjemahkan), *Execute* (jalankan), dan *Store* (simpan).
GUI (*Graphical User Interface*)	Antarmuka pengguna berbasis grafis yang menggunakan ikon, menu, jendela, dan tombol visual untuk memudahkan interaksi manusia dengan komputer.
Interupsi (*Interrupt*)	Sinyal reguler atau kejadian yang menghentikan sementara eksekusi suatu proses di CPU agar prosesor dapat mengalihkan layanan ke tugas lain.
Kuantum (*Time Slice*)	Jatah slot waktu tetap (misalnya 100 ms) yang diberikan secara adil oleh penjadwal (*scheduler*) kepada setiap proses dalam algoritma *Round-Robin*.
Multitasking	Kemampuan Sistem Operasi menjalankan beberapa program aplikasi sekaligus pada satu CPU dengan cara menggilir waktu pemrosesan secara sangat cepat.
Register & Akumulator	Ruang penyimpanan data sementara berkecepatan sangat tinggi yang berada langsung di dalam keping CPU untuk menampung operan dan hasil perhitungan ALU.
Round-Robin (RR)	Algoritma penjadwalan proses di dalam Sistem Operasi yang membagi waktu CPU secara melingkar dan adil tanpa prioritas menggunakan antrean (*Queue*) dan *Kuantum*.
Sistem Operasi (*Operating System*)	Perangkat lunak sistem induk yang mengelola perangkat keras, sumber daya perangkat lunak, memori, berkas, keamanan, serta layanan umum bagi aplikasi komputer.

Daftar Pustaka

- Arduino. (2021). *Arduino Open-Source Electronics Platform*. Diakses dari https://www.arduino.cc/.

- Beekman, G. (2012). *Digital Planet: Tomorrow's Technology and You* (Edisi ke-10). Pearson Prentice Hall.
- Mushthofa, Wahyono, Asfarian, A., Ramadhan, D. A., Putro, H. P., Wisnubhadra, I., Pratiwi, H., & Saputra, B. (2021). *Informatika untuk SMA Kelas X*. Pusat Kurikulum dan Perbukuan, Badan Penelitian dan Pengembangan dan Perbukuan, Kementerian Pendidikan, Kebudayaan, Riset, dan Teknologi Republik Indonesia.
- Operating System. (2021). Dalam *Wikipedia, The Free Encyclopedia* (https://en.wikipedia.org/wiki/Operating_system) & TutorialsPoint (https://www.tutorialspoint.com/operating_system/os_overview.htm).
- Queue (Abstract Data Type). (2021). Dalam *Wikipedia, The Free Encyclopedia* (https://en.wikipedia.org/wiki/Queue_(abstract_data_type)).
- Ramadhan, D. A., Asfarian, A., & Mushthofa. (2021). Algoritma dan Pemrograman. Dalam Mushthofa dkk., *Informatika untuk SMA Kelas X* (Bab 7, hlm. 135–196). Kemendikbudristek.
- Raspberry Pi Foundation. (2021). *Teach, Learn, and Make with Raspberry Pi*. Diakses dari https://www.raspberrypi.org/.
- Vic, F.-W. (2005). *How Computers Work: The CPU and Memory*. University of Rhode Island (https://homepage.cs.uri.edu/faculty/wolfe/book/Readings/Reading04.htm).
- Wahyono, Wisnubhadra, I., & Pratiwi, H. (2021). Sistem Komputer: Komputer, Interaksi Manusia dan Komputer, Sistem Operasi, dan Mesin Konseptual. Dalam Mushthofa dkk., *Informatika untuk SMA Kelas X* (Bab 4, hlm. 63–90). Kemendikbudristek.